Risk Mitigation and Analysis Document

Design and Architecture of an End-to-End Event Ticketing Platform

Document Information

Item	Description
Project Title	Design and Architecture of an End-to-End Event Ticketing Platform
Document Type	Risk Mitigation and Analysis Document
Course	Software Engineering II
System Type	Web-based event ticketing platform
Main Focus	Risk identification, risk analysis, mitigation strategies, monitoring, testing, and response planning

---

Introduction .1

This document analyzes the main risks of the End-to-End Event Ticketing Platform. Since this system deals with real-time seat selection, temporary seat locking, prevention of simultaneous seat reservation, online payment, ticket issuance, and high-traffic management, risk analysis is a very .important part of the project

In such a system, even a small error can cause serious problems. For example, if two users can purchase the same seat at the same time, user trust and organizer confidence will be damaged. Also, if a payment is successful but the ticket is not issued, the system will face a serious consistency .problem

Therefore, the technical, operational, security, and business risks of the system must be identified .from the beginning, and suitable solutions must be designed for each risk

The purpose of this document is to identify the most important possible risks of the system and .provide mitigation strategies, control methods, and response plans for each one

---

Risk Evaluation Method .2

:For risk analysis, each risk is evaluated from two main perspectives

.Probability of occurrence.1

.Impact on the system.2

.The probability level can be Low, Medium, or High
.The impact level can be Low, Medium, High, or Critical

Risks with high probability and high impact must be given the highest priority during design and .implementation

In this project, the most important risks are related to double-booking, payment failure, high traffic, .locked seats not being released, and inconsistency between services

Risk Summary Table .3

.No	Risk	Probability	Impact	Priority Level
1	Simultaneous reservation of the same seat by multiple users	Medium	Critical	Very High
2	Seats remaining locked after timeout or failed payment	High	High	Very High
3	Payment gateway failure or timeout	Medium	High	High
4	Very high traffic during popular event sales	High	Critical	Very High
5	Redis failure or loss of seat locks	Medium	High	High
6	Inconsistency between the main database and seat status	Medium	Critical	Very High
7	Message broker failure, such as RabbitMQ or Kafka failure	Medium	Medium	Medium
8	SMS or email confirmation not being sent	Medium	Medium	Medium
9	API Gateway slowdown or downtime	Medium	High	High
10	Error in QR Code or final ticket generation	Low	High	High
11	Security attacks and unauthorized access	Medium	Critical	Very High
12	Weak monitoring and late error detection	Medium	High	High

Main Risk Analysis .4

Risk of Simultaneous Reservation of the Same Seat 4.1

One of the most important risks in this project is double-booking. This problem happens when two or more users select the same seat within a short time and the system fails to ensure that the seat is .reserved for only one user

Possible Causes

.Lack of a proper locking mechanism•

.Delay between the frontend and backend•

.Multiple concurrent requests for the same seat•

.Weak database transaction design•

.Lack of suitable constraints in PostgreSQL•

Impact on the System

This risk has a critical impact because it can cause the same seat to be sold to multiple users. This is .a serious technical problem and also a major business trust issue

Risk Mitigation Strategy

To reduce this risk, the system must use a temporary seat locking mechanism. When a user selects a seat, the seat should be locked in Redis for a limited time, such as 10 minutes. During this time, .other users must not be able to select the same seat

In addition, PostgreSQL must include a proper database constraint to prevent duplicate final ticket records. Even if an error happens at the application level, the database should not allow two tickets .to be created for the same event and seat

Response Plan

If double-booking occurs, the system must detect it quickly and review the conflicting purchases. Only the first valid purchase should be finalized. The later conflicting purchases should either be .refunded or the users should be offered alternative seats

---

4.2 Risk of Seats Remaining Locked After Failed Payment or Timeout

In the ticketing system, the user first selects a seat and then proceeds to payment. The user may not complete the payment, may close the page, or the payment gateway may return an error. If the seat is not released in this situation, it may remain unavailable for a long time even though it has not .been sold

Possible Causes

- .No TTL for seat locks
- .Error in the payment gateway callback
- .Communication failure between the payment service and reservation service
- .Background worker for releasing expired seats not running
- .Message broker failure

Impact on the System

This problem reduces ticket sales capacity. A seat may not be sold, but it also remains unavailable for other users. In popular events, this can cause serious dissatisfaction for both users and .organizers

Risk Mitigation Strategy

Every seat lock must have a clear TTL. For example, each seat should only be locked for 10 minutes. If the payment is not completed during this period, Redis should automatically remove the .lock

In addition, a background worker should periodically check expired reservations and update their .status in PostgreSQL

Response Plan

If some seats remain locked incorrectly, the system should run a compensation job. This job should .find expired reservations, mark them as expired, and make the related seats available again

---

Risk of Payment Gateway Failure or Timeout 4.3

Payment is one of the most sensitive parts of the system. The external payment gateway may .become slow, become unavailable, fail to send a callback, or return an unclear transaction result

Possible Causes

.External payment service outage•

.Network delay•

.Callback timeout•

.Invalid transaction token•

.Unclear response from the payment gateway•

Impact on the System

If the payment is completed but the system does not detect it, the user may pay money but not receive a ticket. If the payment fails but the system marks it as successful, the system may issue an .incorrect ticket. Both situations are serious and dangerous

Risk Mitigation Strategy

To manage this risk, the system should use the Saga pattern and compensation logic. The payment .process should not depend only on one simple gateway response

The system should store the payment status with a reference ID. If the payment result is unclear, the .system should send an inquiry request to the payment gateway to confirm the final status

Response Plan

If the payment status is unclear, the order should remain in Pending status. A worker should later check the transaction status from the payment gateway. If the payment is successful, the ticket .should be issued. If the payment fails, the seat should be released

---

Risk of Very High Traffic During Popular Event Sales 4.4

During the sale of tickets for a popular event, thousands of users may enter the system at the same time. If the system is not ready for this situation, the backend, database, or Redis may become .overloaded, and the whole service may become unavailable

Possible Causes

.Lack of rate limiting•

.Lack of a virtual waiting room•

.Weak caching strategy•

.Direct pressure from all users on the reservation service•

.Lack of auto-scaling•

.Poorly optimized database queries•

Impact on the System

This risk can cause severe slowness, high error rates, service crashes, and user dissatisfaction. In the .worst case, the ticket sale process may completely fail

Risk Mitigation Strategy

To control this risk, the system should use a virtual waiting room. Users should first enter the waiting room, and the system should allow only a limited number of users to enter the purchasing .process during each time interval

The API Gateway should also include rate limiting. Public event information should be cached to .reduce pressure on the database. Services should also be designed to scale in Kubernetes

Response Plan

If traffic becomes higher than the allowed limit, the system should move new users to the waiting room or display a proper message. Monitoring should detect increased latency and error rate, and .auto-scaling should be activated when necessary

---

Risk of Redis Failure or Loss of Seat Locks 4.5

Redis is used in this project for temporary seat locking. If Redis fails or the locks are lost, the seat .status may become inconsistent

Possible Causes

.Redis crash•

.Memory shortage•

.Lack of persistence or replication configuration•

.Network failure between the reservation service and Redis•

.Accidental deletion of keys•

Impact on the System

This risk can cause locks to disappear or seat status to become incorrect. As a result, multiple users .may be able to select the same seat, or seats may be shown as available incorrectly

Risk Mitigation Strategy

Redis must be configured properly. In a production environment, Redis should preferably be used as a managed or replicated service. The final reservation status must be stored in PostgreSQL because .Redis should only be used for temporary state

Response Plan

If Redis becomes unavailable, the system should temporarily stop new seat selections and display a suitable message to users. After Redis becomes available again, the system should compare lock .data with PostgreSQL records and fix the seat status

---

4.6 Risk of Inconsistency Between the Main Database and Seat Status

In this system, seat status may exist in multiple places: Redis for temporary locks, PostgreSQL for reservations and final tickets, and the frontend for displaying seat status to users. If these parts are .not synchronized, incorrect seat status may be shown

Possible Causes

.Transaction failure•

.Communication failure between services•

.Incomplete message processing•

.Rollback errors•

.Frontend seat status not being updated•

Impact on the System

This problem may cause a sold seat to still appear available, or an available seat to incorrectly .appear locked

Risk Mitigation Strategy

PostgreSQL should be the main source of truth for final seat status. Redis should only be used for temporary locks. Events related to seat status changes should be sent through the message broker so .that other services can update their data

Response Plan

If inconsistency is detected, the system should run a synchronization job. This job should check .tickets, reservations, and locks, then correct the final seat status

---

Risk of Message Broker Failure 4.7

A message broker such as RabbitMQ or Kafka is used for asynchronous communication. For example, when a payment is successful, a message may be sent to issue a ticket or send a .notification. If the message broker fails, these processes may be delayed or interrupted

Possible Causes

.Broker crash•

.Queue becoming full•

.Messages not being processed by workers•

.Consumer errors•

.Message loss•

Impact on the System

Tickets may be issued late, SMS or email notifications may not be sent, or order status may not be .updated on time

Risk Mitigation Strategy

Messages should be durable so they are not lost after broker restart. The system should also include .retry mechanisms and a dead-letter queue for failed messages

Response Plan

If the message broker becomes unavailable, services should temporarily store messages or keep orders in Pending status. After the broker becomes available again, messages should be processed .again

---

Risk of SMS or Email Confirmation Not Being Sent 4.8

After a successful purchase, the user expects to receive an SMS or email confirmation. If the .notification service fails, the user may think that the purchase was not completed

Possible Causes

.SMS provider failure•

.Email provider failure•

.Incorrect email address or phone number•

.Failure in the notification worker•

.Timeout in the sending service•

Impact on the System

This problem usually does not destroy the main data, but it can cause user confusion and .dissatisfaction

Risk Mitigation Strategy

The system should record the delivery status of each notification. If sending fails, the system should .retry. The user should also be able to view and download the ticket from the user panel

Response Plan

If a notification fails, the purchase should not be cancelled. The message should be placed back into .the queue and retried later

---

Risk of Security Attacks and Unauthorized Access 4.9

Since the system deals with user information, payment data, and tickets, security is a very important .concern

Possible Causes

.Weak authentication•

.Poor JWT management•

.Lack of role-based access control•

.SQL Injection attacks•

.XSS attacks•

.Brute-force attacks•

.Exposure of sensitive information•

Impact on the System

This risk can cause unauthorized access, data theft, modification of event information, or fake ticket .generation

Risk Mitigation Strategy

The system should use secure authentication, JWT with proper expiration time, role-based access .control, input validation, password hashing, and rate limiting for suspicious requests

Response Plan

If an attack is detected, suspicious accounts should be limited, active tokens should be revoked, logs .should be reviewed, and the vulnerability should be fixed immediately

---

Risk of Error in QR Code or Final Ticket Generation 4.10

After successful payment, the system must issue the final ticket with a unique QR Code. If the QR .Code is incorrect or duplicated, ticket validation will become problematic

Possible Causes

.Duplicate token generation•

.QR generation error•

.Ticket metadata not being saved•

.Incomplete communication between the payment service and ticket service•

.Validation error at the event entrance•

Impact on the System

This problem may cause the user to receive an unusable ticket or face problems when entering the .event

Risk Mitigation Strategy

Each ticket should have a unique identifier and a secure hash. The QR Code should represent only one valid ticket. The ticket service should verify payment and reservation status before issuing the .ticket

Response Plan

If the QR Code is generated incorrectly, the system should allow reissuing the ticket only for the .same valid purchase and without creating a duplicate ticket

---

Operational Risks and Monitoring .5

To reduce operational problems, the system must include proper monitoring. Tools such as Prometheus and Grafana can be used to monitor service status, resource usage, error rate, latency, .and queue status

:Important monitoring metrics include

.Number of requests per second•

.Average API response time•

.Number of 4xx and 5xx errors•

.Redis status•

.PostgreSQL status•

.Queue length•

.Number of pending payments•

.Number of expired reservations•

.Number of failed notifications•

.CPU and memory usage•

If these metrics are not monitored correctly, the development team may detect problems too late, .and a small issue may become a major incident

---

Test Plan for Risk Reduction .6

.To make sure the system works correctly, the following tests should be considered

Concurrency Test 6.1

In this test, multiple simultaneous requests are sent for the same seat. The correct result is that only .one user should be able to lock or book the seat, and all other requests should be rejected

---

Reservation Timeout Test 6.2

In this test, a seat is selected but payment is not completed. After the TTL expires, the system .should release the seat and make it available again

---

Failed Payment Test 6.3

In this test, payment is intentionally marked as failed. The system should cancel the reservation and .make the seat available again

---

Message Broker Failure Test 6.4

In this test, the message broker is temporarily stopped. The system should not lose the order, and .after the broker becomes available again, messages should be processed again

---

Load and Stress Test 6.5

In this test, many users enter the system at the same time. The goal is to evaluate the behavior of the .virtual waiting room, API Gateway, Redis, and database under pressure

---

Security Test 6.6

In this test, security cases such as unauthorized access, invalid tokens, incorrect roles, malicious .input, and attempts to access other users' data are checked

---

Risk Prioritization .7

:In this project, three risks must have the highest priority

.Double-booking.1

.Payment inconsistency.2

.Flash traffic during popular events.3

The reason for this priority is that all three directly affect user trust and the business success of the system. If the system cannot prevent duplicate seat sales or cannot manage payment correctly, the .product will not be reliable even if the user interface looks good

---

Conclusion .8

Risk analysis is very important in this project because the Event Ticketing Platform deals with sensitive and concurrent operations. The most important risks include simultaneous reservation of the same seat, seats remaining locked, payment failure, high traffic, inconsistency between services, .message broker failure, and security issues

To reduce these risks, the system should use Redis for temporary locking, PostgreSQL for main data, accurate database transactions, the Saga and compensation pattern for payment, a message broker for asynchronous communication, a virtual waiting room for traffic control, and monitoring .tools for fast error detection

By applying these strategies, the system can become more reliable, more stable, and better prepared .for real-world conditions